

7.1. Objetos y clases

Cuando trabajamos con **comunicación asíncrona**, las aplicaciones web no solo envían y reciben datos, sino que necesitan **organizar, interpretar y manipular esa información** de forma eficiente una vez llega al navegador.



Los datos que se reciben del servidor suelen llegar en formatos como **JSON** (JavaScript Object Notation), que representan información estructurada (usuarios, productos, mensajes, publicaciones, etc.).

Para poder trabajar correctamente con estos datos en JavaScript, es necesario transformarlos en **objetos**, que nos permitan acceder a sus propiedades y métodos de forma clara y ordenada.

Las **clases** permiten ir un paso más allá, ya que nos ayudan a:

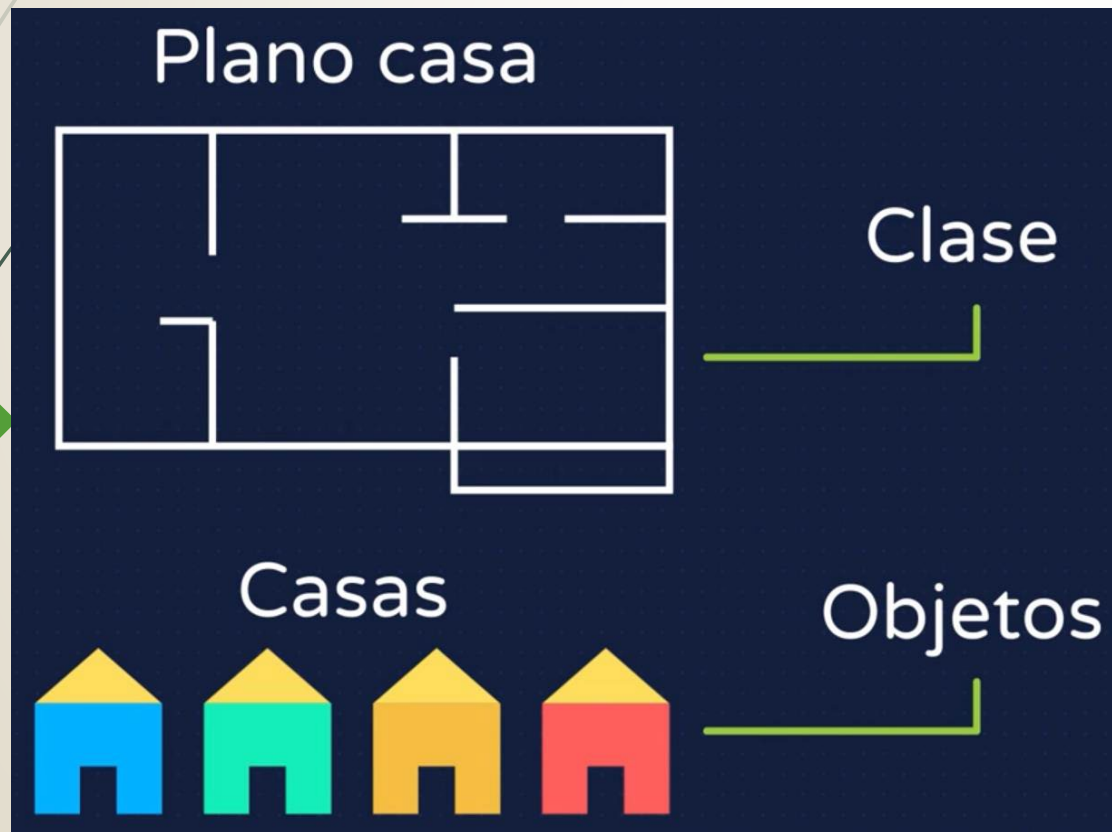
- Modelar entidades reales de la aplicación (Usuario, Producto, Mensaje...)
- Reutilizar código
- Mantener una estructura clara y mantenible
- Separar datos y comportamiento

En una aplicación con comunicación asíncrona, los datos:

1. Se solicitan al servidor (AJAX)
2. Llegan en formato JSON
3. Se convierten en objetos JavaScript
4. Se procesan y se muestran en la interfaz

Sin el uso de objetos y clases, el código se vuelve desordenado, difícil de mantener y poco escalable, especialmente cuando la aplicación crece.

JavaScript es orientado a objetos y, desde su versión ES2015, dispone de una sintaxis más familiar para programadores que conozcan Java, C#, PHP; disponiendo de métodos similares para trabajar con clases, herencia, etc.



```
class Casa {  
  constructor(color, habitaciones) {  
    this.color = color;  
    this.habitaciones = habitaciones;  
  }  
  
  describir() {  
    return `Casa de color ${this.color}`  
  }  
}  
  
const casaAzul = new Casa("azul", 3);  
const casaVerde = new Casa("verde", 2);  
  
console.log(casaAzul.describir());  
console.log(casaVerde.describir());
```

Aunque en JavaScript definiremos “clases”, por dentro el lenguaje no usa clases de verdad.

JavaScript funciona con **prototipos**, y cuando creamos una clase, el motor la convierte automáticamente en un sistema de prototipos.

El sistema de prototipos es la forma en que JavaScript **comparte propiedades y métodos entre objetos**.



JavaScript no copia métodos en cada objeto: los comparte usando prototipos.

Internamente, JavaScript traduce estas clases al sistema basado en **prototipos** que usa en realidad.

Se conoce como **instanciar una clase, crear un objeto o crear una instancia** a la acción de crear un **nuevo objeto** basado en una clase particular.

Esta acción la realizamos a través de la palabra clave **new**, seguida del nombre de la clase, la cuál puede tener parámetros (valores de las **propiedades**), en cuyo caso se controlarían desde un **constructor**.

```
const casaAzul = new Casa("azul", 3);
```

Para crear un constructor de una clase, tenemos que implementar un método llamado **constructor()** que será llamado automáticamente cuando se instancie un nuevo objeto.

```
class Casa {  
    constructor(color, habitaciones) {  
        this.color = color;  
        this.habitaciones = habitaciones;  
    }  
}
```

Cuando se declaran métodos de una clase, internamente se añaden al prototype de la función constructora.

```
class Casa {  
    constructor(color, habitaciones) {  
        this.color = color;  
        this.habitaciones = habitaciones;  
    }  
  
    describir() {  
        return `Casa de color ${this.color} con ${this.habitaciones} habitaciones`;  
    }  
}
```

7.1.1. Prototype

Es una propiedad (y también un objeto) que se incluye por defecto en los objetos de JavaScript.

Solo es accesible directamente desde el constructor.

La mayoría de los navegadores nos permite acceder al prototype a través de la propiedad llamada **__proto__**

Puede ser entendida como un tipo de **herencia**. Cuando buscamos por una propiedad o método se busca en el objeto. Si no está ahí, se busca en su propiedad Prototype, y si no está ahí, buscamos en el Prototype del Prototype, etc. (hasta encontrarnos con undefined).

```
const casaAzul = new Casa("azul", 3);
```

```
console.log(casaAzul.__proto__ === Casa.prototype); // true
```

El objeto **casaAzul** apunta con **__proto__** al **Casa.prototype**.

Si hacemos ...



```
console.log(casaAzul.hasOwnProperty("color"));           // true  
console.log(casaAzul.hasOwnProperty("habitaciones"));  // true  
console.log(casaAzul.hasOwnProperty("describir"));     // false
```

Vemos que:

- las propiedades viven en el objeto y
- los métodos en el prototype

Esto siempre es así cuando definimos métodos en una **class**. Ventajas:

- los métodos se comparten
- menor consumo de memoria
- código más limpio y mantenible
- comportamiento coherente entre objetos

Esquema mental:

`Casa.prototype`

└─ `describir()`

`casaAzul`

└─ `color: "azul"`

└─ `habitaciones: 3`

└─ `__proto__` ──────────▶ `Casa.prototype`

`console.log(casaAzul.describir());`



1. Busca `describir` en `casaAzul`
2. No lo encuentra
3. Va a `casaAzul.__proto__` (→ `Casa.prototype`)
4. Ejecuta el método

¿Podemos tener métodos que “vivan” en el objeto y no en el prototype?

Sí, los definidos en el constructor ...



```
class Casa {  
  constructor(color) {  
    this.color = color;  
  
    this.describir = function () {  
      return this.color;  
    };  
  }  
}  
  
const casa = new Casa("azul");  
  
console.log(casa.hasOwnProperty("describir")); // true
```

En la mayoría de los casos **no hace falta usar prototipos directamente.**

¿Para qué existen los prototipos?

Son el **mecanismo interno** de JavaScript para compartir métodos.

Las class y los objetos “normales” ya los usan por nosotros.



Trabajaremos con class, new, métodos, etc., y todo funcionará.

Es como el motor de un coche: está ahí, pero no lo tocas para conducir.

7.1.2. Propiedades y métodos

A la hora de encapsular el código de las clases es importante el uso de este tipo de elementos.

Tradicionalmente las **propiedades** acostumbraban a **definirse** a través del constructor, mediante la palabra clave **this**; pero desde la versión ES2020 esto se ha visto simplificado (campos de clase).

Antes ...

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
}
```

Ahora ...

Para asignar valores por defecto ...

```
class Persona {  
    nombre = "Anónimo";  
    edad = 0;  
}
```

```
class Persona {  
    nombre;  
    edad = 0;  
  
    constructor(nombre) {  
        this.nombre = nombre;  
    }  
}
```

Para definir métodos como funciones flecha ...

```
class Contador {  
  valor = 0;  
  
  incrementar = () => {  
    this.valor++;  
  };  
}
```

Las propiedades y/o métodos siempre van a tener una **visibilidad** específica, que puede ser **pública** (por defecto) o **privada**.

Pública → Las propiedades y/o métodos pueden ser leídos o modificados tanto desde dentro de la clase como desde fuera.

Privada → Sólo pueden ser leídos o modificados desde el interior de la clase.

Para que una propiedad y/o método sea **privado**, simplemente le añadimos el **símbolo #** antes del nombre.

Propiedad privada ...

```
class Usuario {  
    #password;  
  
    constructor(password) {  
        this.#password = password;  
    }  
}
```



Ejemplo más completo:

Uso correcto:

```
const cuenta = new CuentaBancaria(100);  
cuenta.depositar(50);  
console.log(cuenta.obtenerSaldo()); // 150
```

Uso incorrecto:

```
cuenta.#saldo; // ❌ Error de sintaxis  
cuenta.#validarSaldo(5); // ❌ Error de sintaxis
```

```
class CuentaBancaria {  
    #saldo = 0;  
  
    constructor(saldoInicial) {  
        this.#validarSaldo(saldoInicial);  
        this.#saldo = saldoInicial;  
    }  
  
    // método privado  
    #validarSaldo(cantidad) {  
        if (cantidad < 0) {  
            throw new Error("El saldo no puede ser negativo");  
        }  
    }  
  
    // método público  
    depositar(cantidad) {  
        this.#validarSaldo(cantidad);  
        this.#saldo += cantidad;  
    }  
  
    // método público  
    obtenerSaldo() {  
        return this.#saldo;  
    }  
}
```

Propiedades computadas

Son un tipo de propiedad especial que se declaran como una función, y que se ejecuta cuando accedemos a la propiedad con dicho nombre.

Hay dos tipos de propiedades computadas, los **getters** y los **setters**.



→ Se usan para leer o modificar propiedades privadas o en propiedades públicas sólo si aportan lógica.

No harían falta ...

```
class Producto {  
    precio;  
  
    constructor(precio) {  
        this.precio = precio;  
    }  
}
```

```
const p = new Producto(100);  
console.log(p.precio); // 100
```

Sí, por ejemplo, para añadir lógica ...

```
class Producto {  
    precio; // propiedad pública
```

```
// Setter para añadir lógica
```

```
    set precio(valor) {  
        if (valor <= 0) {  
            throw new Error("Precio inválido");  
        }  
        this._precio = valor; // usamos propiedad interna _precio para evitar recursión  
    }
```

Antes de ES2020: _ (convención)

```
class Usuario {  
  constructor(edad) {  
    this._edad = edad; // _ indica "privada" por convención  
  }  
  
  get edad() {  
    return this._edad;  
  }  
  
  set edad(valor) {  
    if (valor < 0) throw new Error();  
    this._edad = valor;  
  }  
}
```

```
u._edad = -100; // ✗ aunque no debería
```

_edad no es realmente privada

se usaba _ solo para indicar "no tocar desde fuera"

se confiaba en que otros devs respetaran la convención

Desde ES2020: # (privado real)

```
class Usuario {  
  #edad;  
  
  constructor(edad) {  
    this.#edad = edad;  
  }  
  
  get edad() {  
    return this.#edad;  
  }  
  
  set edad(valor) {  
    if (valor < 0) throw new Error();  
    this.#edad = valor;  
  }  
}
```

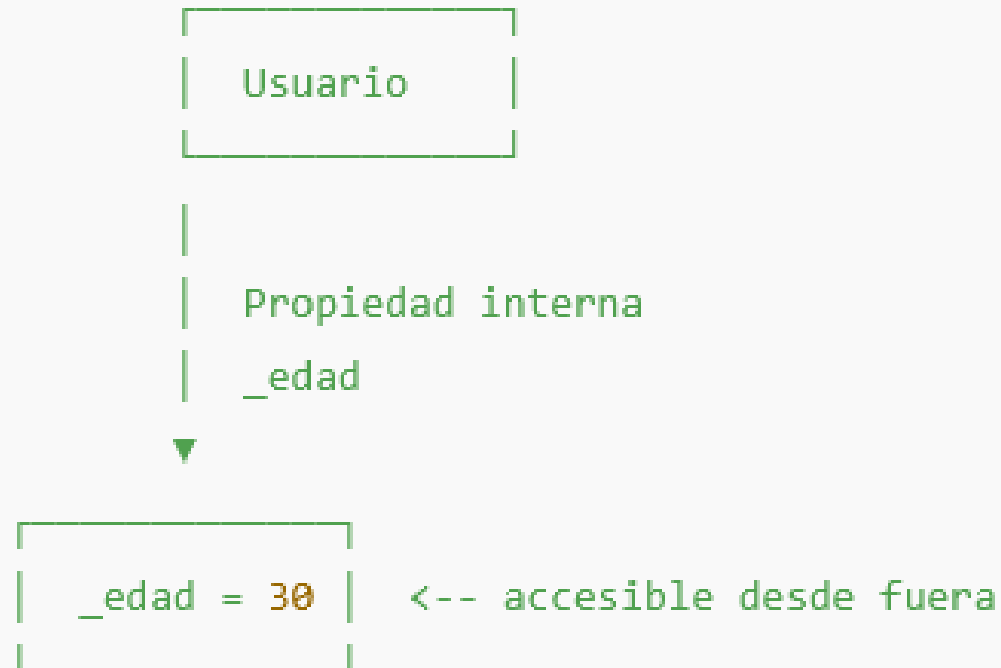
```
u.#edad = 30; // ✗ SyntaxError  
console.log(u.#edad); // ✗ SyntaxError
```

#edad es
verdaderamente
privada

no hay forma de
saltarse la
encapsulación

getter y setter son
la única forma de
leer o escribir (si los
defines)

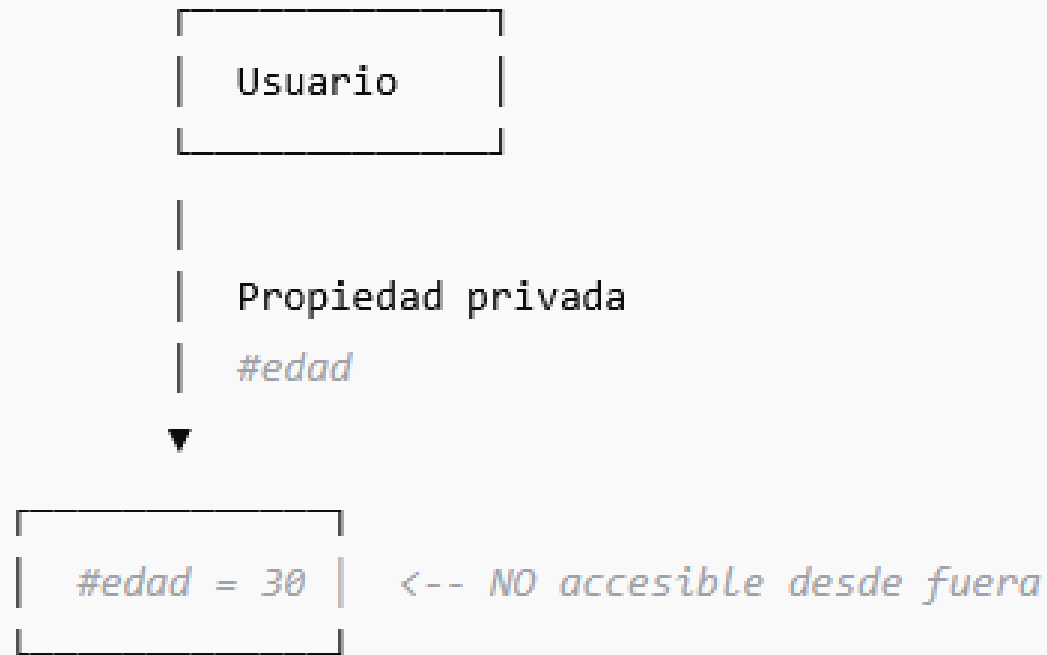
ANTES: Convención con _



Getter/Setter:

usuario.edad = 40 ← Llama al setter, pero alguien podría hacer:
usuario._edad = -100 ← ✗ rompe la “privacidad”
console.log(usuario.edad) ← llama al getter

AHORA: Privado real con #



Getter/Setter:

usuario.edad = 40 ← Llama al setter y valida

console.log(usuario.edad) ← llama al getter

usuario.#edad = 50 ← **X** SyntaxError, imposible acceder directamente

Ejemplo final “moderno”

```
class Producto {  
  #precio; // propiedad privada real  
  
  constructor(precioInicial) {  
    this.precio = precioInicial; // usa el setter  
  }  
  
  // Setter con validación  
  set precio(valor) {  
    if (valor <= 0) {  
      throw new Error("Precio inválido");  
    }  
    this.#precio = valor;  
  }  
  
  // Getter  
  get precio() {  
    return this.#precio;  
  }  
  
  // Método adicional opcional  
  aplicarDescuento(porcentaje) {  
    this.precio = this.#precio * (1 - porcentaje / 100);  
  }  
}
```

```
const p = new Producto(100);  
console.log(p.precio); // 100 → getter  
  
p.precio = 150; // setter  
console.log(p.precio); // 150  
  
p.aplicarDescuento(10);  
console.log(p.precio); // 135
```

Propiedades y métodos estáticos

Son atributos y funciones de una clase, pero que no requieren crear una instancia para ejecutarlos, sino que se pueden ejecutar directamente haciendo referencia al nombre de la clase.



```
class Empleado {
  static rol = "NiNi"; // propiedad pública estática
  static #sueldoMinimo = 150000; // propiedad privada estática

  constructor(nombre, sueldo, rol) {
    this.nombre = nombre;
    this.sueldo = sueldo;
    this.rol = rol;
  }

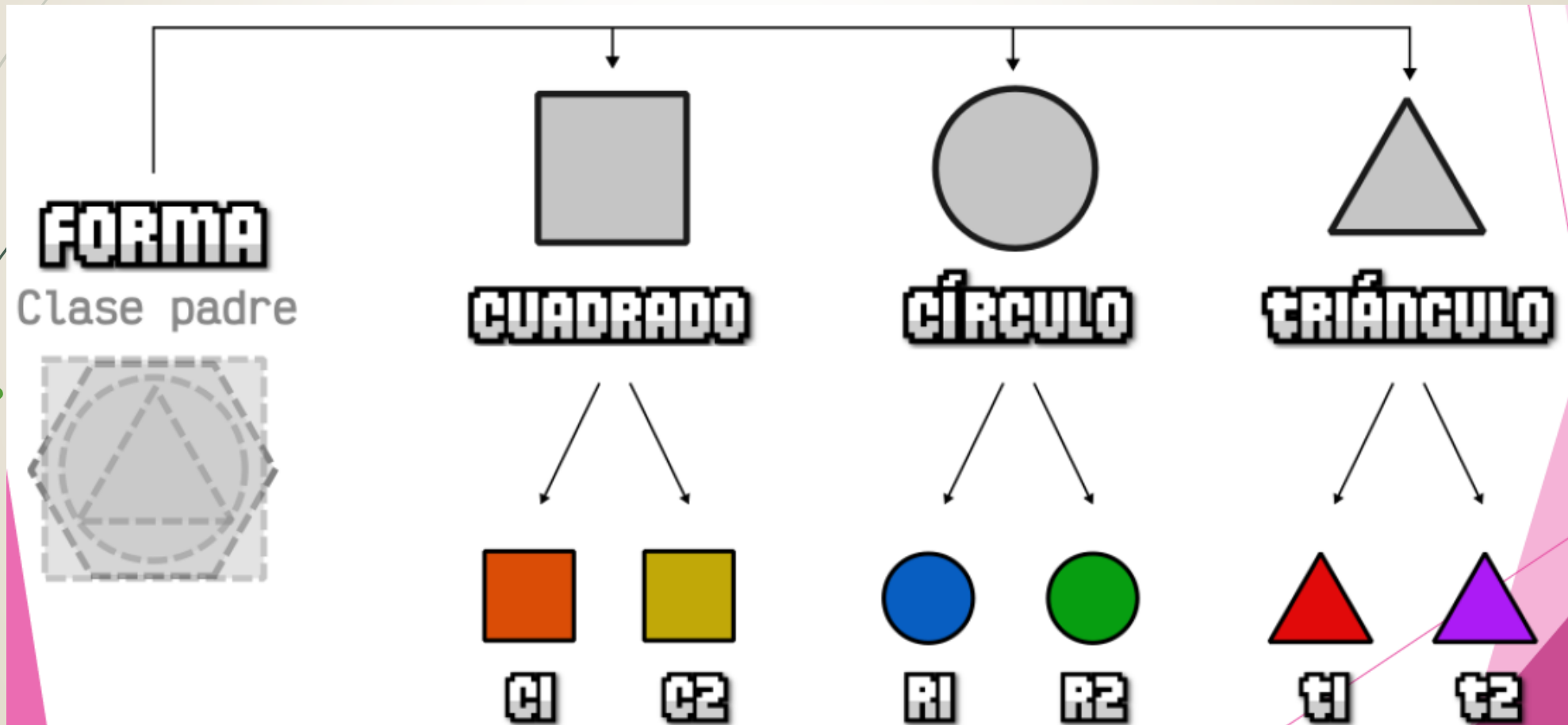
  // todavía no se ha ejecutado el constructor
  // y se puede referenciar la propiedad sueldoMinimo internamente
  // en este método estático
  static creaBecario(nombre, rol) {
    return new Empleado(nombre, Empleado.#sueldoMinimo, rol);
  }
}
```

```
console.log(Empleado.rol); // se puede referenciar la propiedad rol desde fuera
```

```
let empleada = Empleado.creaBecario("Juanan", Empleado.rol = 'Profesor'); // referencia
console.log(empleada); // Object { nombre: "Juanan", sueldo: 150000, rol: "Profesor" }
```

7.1.3. Herencia

Una de las características fundamentales que nos ayudan a **reutilizar y simplificar código** es la herencia de Clases.



La herencia es la característica donde **una clase hija obtiene las propiedades y métodos de una clase padre** porque se ha establecido una relación entre ambas.

En JavaScript, a partir de ES2015, podemos «**extender clases**» de forma muy similar a como se hace en otros lenguajes de programación; a través de la palabra clave **extends**.



```
/**
 * Clase PADRE
 */
class Persona {
    constructor(nombre, edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    saluda() {
        console.log(`Hola, soy ${this.nombre} y tengo ${this.edad} años`);
    }

    toString() {
        return `${this.nombre} (${this.edad})`;
    }
}
```

```
/**
 * Clase HIJA
 * Extiende de su clase padre, por lo que la clase Usuario será una mezcla de lo que
 * tiene la clase Persona más lo que tiene la clase Usuario.
 */
class Usuario extends Persona {
    constructor(nombre, edad, email, password) {
        // super(): Función especial que llama al constructor de la clase padre;
        // por lo que antes de continuar, pasa a ejecutarse el constructor de Persona
        super(nombre, edad);
        this.email = email;
        this.password = password;
    }
    // Comentar y descomentar
    saluda() {
        // super.saluda(); // llamada al constructor de saluda() de Persona
        console.log(`Mi correo es ${this.email}`);
    }
}
```

```
const user = new Usuario("Pepe", 23, "pepe@email.com", "1234");  
console.log(user); // Object { nombre: "Pepe", edad: 23, email: "pepe@email.com", password: "1234" }  
user.saluda(); // 'Hola, soy Pepe y tengo 23 años' | 'Mi correo es pepe@email.com'  
console.log(`Datos usuario: ${user}`); // toString() de Persona --> no sobrecargada en Usuario
```

7.1.4. Valor primitivo y valor string

Cuando un objeto es convertido a string (concatenándolo, por ejemplo), el método **toString** (heredado de Object) es automáticamente llamado.

Por defecto, imprimirá el tipo de objeto "[object Object]", pero podemos **sobreescibir** o **sobrecargar** el comportamiento de este método en el objeto (o en su prototype).

Cuando comparamos objetos usando los operadores relacionales ($>$, $<$, $=>$, $=<$), obtenemos el **valor primitivo** por defecto (por defecto **toString()**).

Si sobrescribimos el método **valueOf()** (heredado de Objeto), devuelve otro valor primitivo, que será usado para este tipo de comparaciones.

```
class Producto {  
    constructor(nombre, precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    // Sobreescribir toString() → cuando conviertas el objeto a cadena  
    toString() {  
        return `${this.nombre} cuesta ${this.precio}`;  
    }  
  
    // Sobreescribir valueOf() → cuando el objeto se use como número  
    valueOf() {  
        return this.precio;  
    }  
}
```

```
const p = new Producto("Camiseta", 25);

// toString()
console.log(String(p)); // "Camiseta cuesta $25"
console.log(`${p}`);    // Template strings también usan toString → "Camiseta cuesta $25"

// valueOf()
console.log(p + 10);    // 25 + 10 → 35, porque valueOf devuelve precio
console.log(p > 20);    // true, porque valueOf devuelve precio
```

toString()

- Se llama automáticamente cuando quieres convertir el objeto a **cadena**.
- Útil para mostrar el objeto de forma legible.

valueOf()

- Se llama cuando quieres usar el objeto como **número** o en operaciones matemáticas.
- Devuelve normalmente un valor primitivo (número, string, boolean).

Ejemplo: Carrito de Compras

```
class Producto {  
    constructor(nombre, precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    toString() {  
        return `${this.nombre} (${this.precio})`;  
    }  
  
    valueOf() {  
        return this.precio;  
    }  
}
```



```
class Carrito {
  constructor() {
    this.items = [];
  }

  agregar(producto) {
    this.items.push(producto);
  }

  // Sumar todos los productos usando valueOf()
  total() {
    return this.items.reduce((sum, p) => sum + p, 0);
  }

  // Mostrar carrito como cadena
  toString() {
    return this.items.map(p => p.toString()).join(", ");
  }
}
```



```
const p1 = new Producto("Camiseta", 25);
const p2 = new Producto("Pantalón", 40);
const p3 = new Producto("Gorra", 15);

const carrito = new Carrito();
carrito.agregar(p1);
carrito.agregar(p2);
carrito.agregar(p3);

console.log(`${carrito}`);
// "Camiseta ($25), Pantalón ($40), Gorra ($15)" → usa toString()

console.log(carrito.total());
// 80 → usa valueOf() de cada Producto

console.log(carrito.total() > 50);
// true
```

`toString()`

- Permite mostrar cada producto o el carrito completo como una cadena legible.
- Se activa con `${obj}` o `String(obj)`.

`valueOf()`

- Permite que los productos se **sumen como números** en operaciones.
- Por eso `reduce((sum, p) => sum + p, 0)` funciona correctamente.

Combinando ambos métodos, puedes tener **objetos que se comporten como números o cadenas** según el contexto.